MODULE 0 · FOUNDATIONS

Storage & the CSI

Turning a PVC into mounted, persistent storage

WHY IT MATTERS

A container's filesystem forgets everything

A Pod's container filesystem is **ephemeral** — it vanishes the moment the Pod dies. Any workload that must survive a restart — a database, a queue, anything stateful — needs storage that lives outside the Pod.

The **Container Storage Interface (CSI)** is the standard plugin API that lets any storage system provision, attach, mount, resize, and snapshot volumes for Kubernetes — without that vendor's code living inside Kubernetes itself.

BY THE END YOU CAN

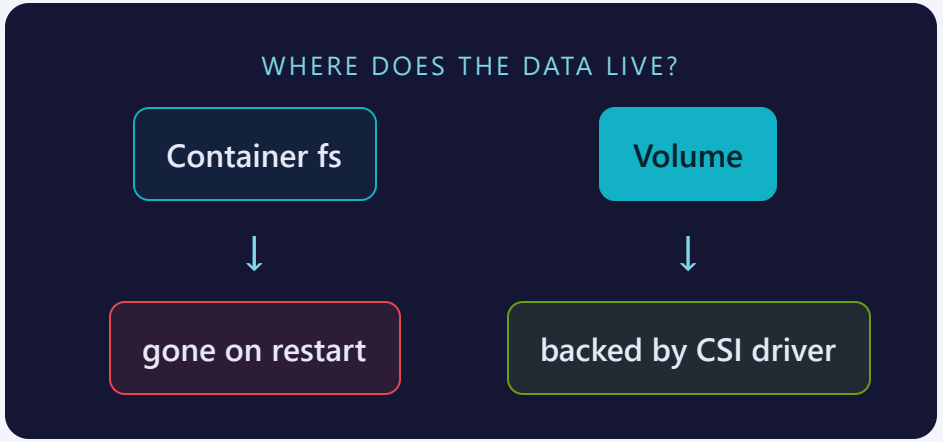
- Explain why CSI replaced in-tree drivers
- Name the two halves of a CSI driver
- Trace a PVC to a mounted volume
- Know where to look when a PVC sticks `Pending`

CONCEPT

Two very different storage stories

- **Container filesystem** — tied to the container's lifetime; gone the instant it's replaced
- **Volume** — backed by real storage: a block disk, a network file share, an on-node path
- CSI is the interface that lets Kubernetes drive *any* of those backends the same way

You already write PVs, PVCs and StorageClasses — CSI is the machinery running underneath them.



THEN AND NOW

In-tree drivers are on the way out

IN-TREE DRIVER (DEPRECATED)	OUT-OF-TREE CSI DRIVER
Compiled into Kubernetes core, e.g. `kubernetes.io/cloud-disk`	Ships as its own image, e.g. `diskplugin.csi.example.com`
Upgrades only on the Kubernetes release cycle	Vendor ships fixes and features on its own schedule
Kubernetes core has to know every backend	Kubernetes core only has to know one interface: CSI

MENTAL MODEL

In-tree volume plugins are deprecated and being removed from core — CSI is the only supported path going forward, for every storage backend.

CSI DRIVER ANATOMY

Two halves, two Kubernetes workload types

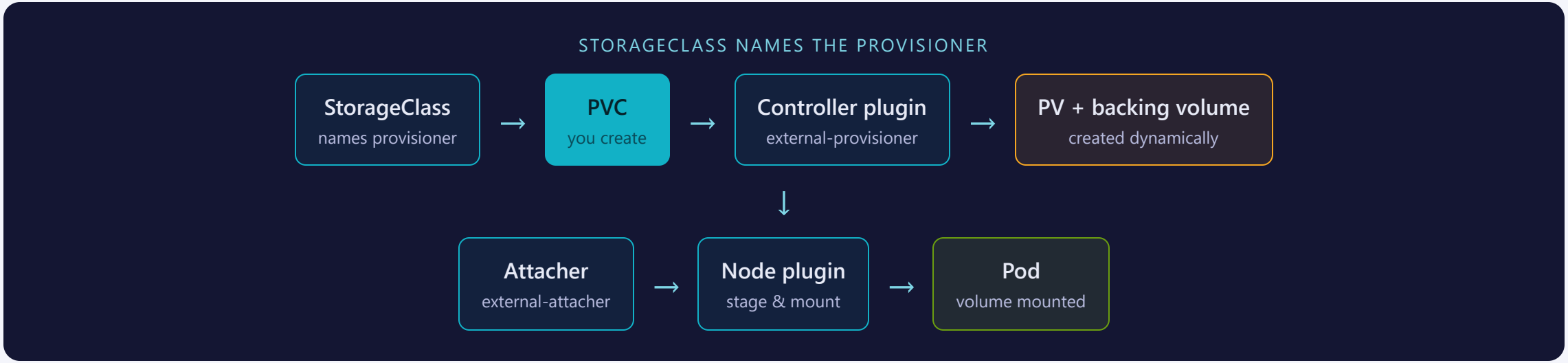
CONTROLLER PLUGIN	NODE PLUGIN
Runs as a `Deployment` — one, cluster-wide	Runs as a `DaemonSet` — one per node
Sidecars: `external-provisioner`, `external-attacher`, `external-resizer`, `external-snapshotter`	Sidecar: `node-driver-registrar`
Talks to the storage backend's own API	Stages and mounts the volume into the Pod

MENTAL MODEL

Both halves implement the same CSI gRPC interface — one centralized workload, one per-node workload, driven by the same driver binary.

DYNAMIC PROVISIONING

From PVC to mounted volume



`kubectl describe pvc <name>` shows this exact sequence as events — provisioning, attach, mount, end to end.

WHAT CSI STANDARDIZES

Same PVC, any backend

OPERATION	HANDLED BY
Provision — create the PV + backing volume	Controller plugin, `external-provisioner`
Attach — attach the volume to a node	Controller plugin, `external-attacher`
Resize — grow the volume	Controller plugin, `external-resizer`
Snapshot — point-in-time copy	Controller plugin, `external-snapshotter` + snapshot CRDs
Mount — stage & mount into the Pod	Node plugin, registered by `node-driver-registrar`

MENTAL MODEL

Same PVC, same YAML — swap the provisioner (`diskplugin.csi.example.com` vs. `local-path`) and every one of these operations targets a different backend.

GOTCHAS — THE ONES THAT BITE

Where people trip

- **Access mode is set by the driver, not by you.** Block storage is usually `RWO`; asking a disk driver for `ReadWriteMany` fails — use a file backend for `RWX`.
- **Disk attach is per-node.** A node can only attach a bounded number of volumes; small nodes hit the limit and new Pods stay unschedulable.
- **Missing or failed CSI driver → PVC stuck `Pending`.** No provisioner means nothing creates the PV — check `kubectl get csidrivers` and the controller Pod's events.
- **Snapshots need extra pieces.** The `external-snapshotter` sidecar and the snapshot CRDs must be installed — they are not part of core Kubernetes.

RECAP

Storage & the CSI in one breath

- Container filesystems are ephemeral; volumes are backed by real storage
- CSI replaced in-tree drivers — one interface, any vendor
- Controller plugin (Deployment) provisions and attaches; node plugin (DaemonSet) mounts
- A `Pending` PVC is almost always a missing driver, not slowness

REMEMBER

PVC → StorageClass names a provisioner → controller creates PV + volume → attacher attaches it → node plugin mounts it.

Next: **1.1 — kubectl & Imperative vs Declarative**